The Wireshark Playbook

A complete, detailed guide to packet capture and traffic analysis with Wireshark — from a single frame to a full incident timeline

From capture filters to TLS handshakes, this playbook builds the packet-level fluency and filter-syntax reference that SOC analysts and network engineers rely on to see exactly what a network is doing.

Author: Swastik Sagar
Final-year Computer Science Undergraduate
SOC Analyst Intern

August 2, 2026

Preface

Every protocol in the networking stack eventually has to put bits on a wire, and Wireshark is the tool that lets you actually look at them. It sits underneath almost every other layer of network and security tooling: a SIEM alert says "suspicious traffic to this host," and Wireshark is what tells you whether that traffic was a TLS handshake to a legitimate service or a malware beacon calling home every sixty seconds.

This playbook treats Wireshark as a working tool, not just a GUI tour. It covers how to capture traffic correctly in the first place, the full capture and display filter syntax, how to read the protocols you'll see most often, and how to recognize the traffic patterns that actually matter during an investigation — ARP spoofing, DNS tunneling, port scans, and more. The final chapters cover `tshark`, Wireshark's command-line counterpart, for when a GUI isn't an option.

What this book covers

- Capturing traffic correctly: interfaces, promiscuous mode, capture options
- Capture filters vs. display filters, and when to use each
- Full display filter syntax, operators, and practical examples
- Following TCP streams and analyzing conversations
- Reading the protocols you'll see constantly: Ethernet, IP, TCP/UDP, HTTP, DNS, TLS
- Statistics tools: I/O graphs, conversations, endpoints, protocol hierarchy
- Recognizing malicious traffic patterns during an investigation
- Exporting objects, packets, and building a report from a capture
- Practical analysis recipes for common SOC scenarios
- Command-line packet analysis with `tshark`

Swastik Sagar

Contents

Preface **i**

1 Introduction to Wireshark **1**

1.1 What Wireshark is . . . 1

1.2 Why packet-level visibility matters . . . 1

1.3 The three-pane interface . . . 1

2 Capturing Traffic **2**

2.1 Choosing an interface . . . 2

2.2 Promiscuous mode . . . 2

2.3 Capturing on a switched network . . . 2

2.4 Capture options . . . 2

3 Capture Filters **4**

3.1 Capture filters vs. display filters . . . 4

3.2 BPF capture filter syntax . . . 4

4 Display Filters **5**

4.1 Basic syntax . . . 5

4.2 Comparison operators . . . 5

4.3 Logical operators . . . 5

4.4 Practical filter examples . . . 6

4.5 Coloring rules . . . 6

5 Following Streams and Conversations **7**

5.1 Follow TCP/UDP/HTTP Stream . . . 7

5.2 Conversations . . . 7

6 Protocol Analysis Deep Dive **8**

6.1 Ethernet and ARP . . . 8

6.2 IP and ICMP . . . 8

6.3 TCP . . . 8

6.4 HTTP . . . 8

6.5 DNS 9
6.6 TLS 9

7 Statistics and Visualization Tools 10
7.1 Protocol Hierarchy 10
7.2 I/O Graphs 10
7.3 Endpoints 10
7.4 Expert Information 10

8 Recognizing Malicious Traffic Patterns 11
8.1 Port scanning 11
8.2 ARP spoofing / MITM 11
8.3 C2 beaconing 11
8.4 Data exfiltration 11

9 Exporting and Reporting 13
9.1 Exporting objects 13
9.2 Exporting packets and filtered subsets 13
9.3 Printing and reporting 13

10 Reading Hex Dumps and the Packet Bytes Pane 14
10.1 Why read raw bytes at all 14
10.2 The Packet Bytes pane, field by field 14
10.3 Decoding a header by hand 14
10.4 Standalone hex tools outside Wireshark 15
10.5 Exporting a packet as a hex dump from Wireshark 15
10.6 Decode As: overriding protocol detection 15

11 Command-Line Analysis with tshark 16
11.1 Why tshark 16
11.2 Basic usage 16
11.3 Applying filters 16
11.4 Extracting specific fields 16
11.5 Output formats 17
11.6 Statistics from the command line 17
11.7 Rolling captures and long-running collection 17
11.8 Combining tshark with standard Unix tools 17

1

Introduction to Wireshark

1.1 What Wireshark is

Wireshark is a free, open-source packet analyzer. It captures traffic passing through a network interface and lets you inspect every layer of every packet — from raw Ethernet frames up through application-layer protocol data — in a structured, readable form instead of raw bytes.

1.2 Why packet-level visibility matters

- **Ground truth** — logs can be incomplete or tampered with; a packet capture shows exactly what crossed the wire
- **Protocol-level detail** — see the actual handshake, headers, and payload, not just a summary
- **Malware analysis** — observe exactly how a sample communicates with its C2 infrastructure
- **Troubleshooting** — diagnose problems no application-level log can explain, like retransmissions or malformed packets

1.3 The three-pane interface

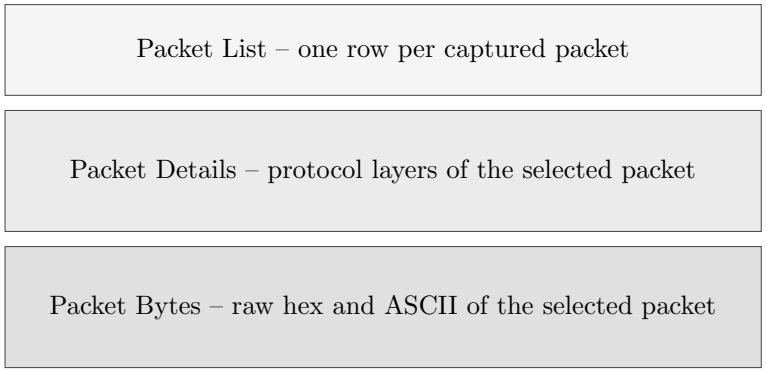

Figure 1.1. *Wireshark's three-pane layout: list, details, and raw bytes, all synchronized to the same selected packet.*

Reading the panes together

Clicking a field in the Packet Details pane highlights the corresponding bytes in the Packet Bytes pane — this is the fastest way to understand exactly which bytes on the wire correspond to which protocol field.

2

Capturing Traffic

2.1 Choosing an interface

Wireshark lists every available network interface at startup, along with a live mini-graph of traffic volume, which is usually enough to identify the right one on a multi-interface machine.

2.2 Promiscuous mode

By default, a network interface only passes up frames addressed to its own MAC address. **Promiscuous mode** tells the interface to pass up every frame it physically receives, regardless of destination — necessary to see other hosts' traffic on a shared segment, though largely irrelevant on modern switched networks where a host normally only receives its own traffic anyway (see port mirroring below).

2.3 Capturing on a switched network

Because switches only forward traffic to the port it's actually destined for, plugging into a random switch port and enabling promiscuous mode won't show you other hosts' traffic. To capture broader traffic on a switch:

- **Port mirroring / SPAN** — configure the switch to copy traffic from other ports to your capture port
- **Network tap** — a physical device inserted inline that duplicates traffic to a monitor port
- **ARP spoofing** — redirects traffic through your own host (active, detectable, and only for authorized testing)

2.4 Capture options

Starting a capture from the command line with dumpcap

```
dumpcap -i eth0 -w capture.pcapng dumpcap -i eth0 -a duration:60 -w
capture.pcapng dumpcap -i eth0 -b filesize:10000 -w rolling.pcapng
```

Option	Purpose
Snapshot length	Caps how many bytes of each packet are captured, saving space if full payloads aren't needed
Ring buffer	Rotates through a fixed set of capture files instead of one endlessly growing file
Stop conditions	Auto-stop capture after a duration, file size, or packet count

File formats

.pcapng is the modern default format and supports richer metadata; **.pcap** is the older, simpler format still expected by some legacy tools. Wireshark reads and writes both.

3

Capture Filters

3.1 Capture filters vs. display filters

	Behavior
Capture filter	Applied *before* capture; matching traffic is the only traffic saved; uses BPF (Berkeley Packet Filter) syntax
Display filter	Applied *after* capture, to traffic already saved; only changes what's shown, doesn't discard data; uses Wireshark's own filter syntax

Which one to use

Capture filters are for reducing capture *volume* on a busy link (you can't change your mind about discarded packets later); display filters are for narrowing your *view* of data you've already captured, and are far more commonly used during actual analysis.

3.2 BPF capture filter syntax

Common capture filters

```
host 192.168.1.10 net 192.168.1.0/24 port 443 tcp port 80 or tcp port 443 not
broadcast and not multicast src host 10.0.0.5 and dst port 22
```

4

Display Filters

4.1 Basic syntax

Display filters reference a protocol or field name, an operator, and (usually) a value.

Basic display filters

```
ip.addr == 192.168.1.10 tcp.port == 443 http.request.method == "GET" dns.qry.name
== "example.com"
```

4.2 Comparison operators

Operator	Alt.	Meaning
==	eq	Equal to
!=	ne	Not equal to
¿	gt	Greater than
¡	lt	Less than
¿=	ge	Greater than or equal
¡=	le	Less than or equal
contains	—	Field contains a substring
matches	—	Field matches a regular expression

4.3 Logical operators

Combining filters

```
ip.addr == 10.0.0.5 and tcp.port == 443 http or dns not arp tcp.flags.syn == 1
and tcp.flags.ack == 0
```

4.4 Practical filter examples

Filter	Shows
`tcp.flags.syn==1 && tcp.flags.ack==0`	Only SYN packets, useful for spotting scans
`tcp.analysis.retransmission`	Retransmitted TCP segments, a sign of packet loss or latency
`http.request`	Only HTTP request packets
`dns.flags.rcode != 0`	DNS responses with an error code
`tls.handshake.type == 1`	TLS ClientHello packets
`frame contains "password"`	Any packet with the literal string "password" in its raw bytes

Right-click to build filters

Right-clicking any field in the Packet Details pane and choosing "Apply as Filter" builds correct filter syntax automatically — often faster than typing filters from memory, especially for unfamiliar protocol fields.

4.5 Coloring rules

Wireshark colors packets by protocol/condition using the same filter syntax — e.g. black-on-red for packets with checksum errors, black-on-yellow for retransmissions — and these rules are fully customizable under *View* > *Coloring Rules*, letting you visually flag anything a display filter can express.

5

Following Streams and Conversations

5.1 Follow TCP/UDP/HTTP Stream

Right-clicking a packet and choosing *Follow* > *TCP Stream* (or UDP/HTTP/TLS Stream) reassembles the full back-and-forth conversation into readable order, with each side's data color-coded separately — far easier than reading individual segments one at a time.

Why this matters

A single HTTP request can span many TCP segments. Following the stream reconstructs the full request and response bodies as they were actually sent, which is usually what you actually want to read rather than individual packet fragments.

5.2 Conversations

Statistics > *Conversations* lists every distinct communication pair (by MAC, IP, or port), along with packet counts, byte counts, and duration — useful for quickly spotting the largest data transfers or longest-lived connections in a capture.

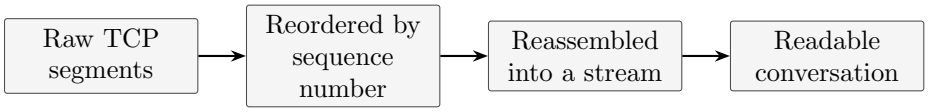

Figure 5.1. *Following a stream does this reassembly automatically, regardless of the order packets actually arrived in.*

6

Protocol Analysis Deep Dive

6.1 Ethernet and ARP

The Packet Details pane's first layer is almost always Ethernet: source and destination MAC, and an EtherType field identifying the next layer. ARP packets are easy to filter (`arp`) and worth watching for — a flood of ARP replies for the same IP with different MACs is a classic spoofing signature.

6.2 IP and ICMP

Useful IP/ICMP filters

```
ip.ttl < 10 ip.flags.mf == 1 icmp.type == 8 icmp.type == 3
```

TTL, fragmentation flags, and ICMP type/code are the fields worth knowing by number: type 8 is an echo request (ping), type 3 is destination unreachable — the latter is what confirms a filtered/closed port during scan analysis.

6.3 TCP

Field	What to look for
`tcp.flags`	SYN/ACK/FIN/RST combinations identify handshake stage or scan type
`tcp.seq` / `tcp.ack`	Sequence tracking; large jumps can indicate loss or out-of-order delivery
`tcp.window_size`	Flow control window; a window of 0 means the receiver's buffer is full
`tcp.analysis.flags`	Wireshark's own annotations: retransmission, duplicate ACK, out-of-order

6.4 HTTP

HTTP-focused filters

```
http.request.method == "POST" http.response.code >= 400 http contains
"User-Agent"
```

For encrypted HTTPS traffic, the HTTP layer itself isn't visible — only the TLS handshake and encrypted application data are, unless you have the session keys (see the TLS note below).

6.5 DNS

DNS-focused filters

```
dns.qry.name contains "suspicious-domain" dns.flags.response == 0 dns.a ==
10.0.0.5
```

DNS tunneling signatures

Unusually long subdomains, high query volume to one domain, and TXT-record- heavy traffic are the classic indicators of DNS being used as a covert data channel rather than for name resolution.

6.6 TLS

Wireshark can decrypt TLS traffic if given the session keys, via *Edit* > *Preferences* > *Protocols* > *TLS* and pointing at a keylog file — most commonly generated by setting the `SSLKEYLOGFILE` environment variable before starting a browser.

Enabling a TLS keylog file, before starting the browser

```
export SSLKEYLOGFILE= /tls-keys.log
```

Without the keys, you can still see useful metadata unencrypted: the ClientHello's SNI (Server Name Indication) field reveals which hostname was requested even before decryption, which is often enough for triage.

7

Statistics and Visualization Tools

7.1 Protocol Hierarchy

Statistics > Protocol Hierarchy breaks down the entire capture by protocol, showing what percentage of traffic is TCP, UDP, HTTP, DNS, and so on — a fast way to get oriented in an unfamiliar capture before digging into individual packets.

7.2 I/O Graphs

Statistics > I/O Graph plots traffic volume over time, optionally split by a display filter per line — ideal for visually spotting a beaconing pattern (regular spikes) or a sudden volume spike (exfiltration or a DoS condition).

7.3 Endpoints

Statistics > Endpoints lists every unique address seen in the capture (Ethernet, IPv4, IPv6, TCP, UDP), with traffic totals per endpoint — useful for quickly identifying the busiest or most unusual host.

Tool	**Best for**
Protocol Hierarchy	Getting oriented in a new, unfamiliar capture
I/O Graph	Spotting timing patterns: beaconing, spikes, outages
Conversations	Finding the largest or longest-lived flows
Endpoints	Identifying every host present and how active it was
Expert Information	Wireshark's own automated warnings and anomaly flags

7.4 Expert Information

Analyze > Expert Information surfaces Wireshark's own built-in anomaly detection — malformed packets, retransmissions, checksum errors — categorized by severity, often the fastest starting point when a capture is large and you don't yet know what you're looking for.

8

Recognizing Malicious Traffic Patterns

8.1 Port scanning

- Many SYN packets to sequential or varied ports on one host, with few completed handshakes
- A high ratio of RST responses to SYN packets sent
- Traffic to many different destination hosts from one source in a short window (horizontal scanning)

Filter for likely scan traffic

```
tcp.flags.syn==1 and tcp.flags.ack==0
```

8.2 ARP spoofing / MITM

- Multiple ARP replies claiming the same IP with different source MACs
- Gratuitous ARP announcements at unusual frequency
- A host suddenly resolving the same IP to a new MAC without any corresponding hardware change

8.3 C2 beaconing

- Highly regular, periodic connections to the same external host, visible clearly in an I/O Graph
- Small, consistent payload sizes on every beacon
- Repeated DNS queries to a domain immediately followed by a connection to the resolved IP

8.4 Data exfiltration

- A single large, sustained outbound transfer to an external or unfamiliar destination
- Unusual protocols/ports carrying large volumes (e.g. large ICMP payloads, DNS TXT record floods)
- Transfers timed outside a host's normal working-hours pattern

Baseline first

None of these patterns are inherently malicious in isolation — a backup job also produces a large sustained transfer. Recognizing an anomaly requires knowing what *normal* looks

like for that host or network first.

9

Exporting and Reporting

9.1 Exporting objects

File > Export Objects pulls files directly out of a capture — HTTP, SMB, and other file-transfer protocols can be reconstructed and saved to disk, which is often how a malware sample or exfiltrated document gets recovered from a pcap during an investigation.

9.2 Exporting packets and filtered subsets

Saving only the filtered/displayed packets

```
-- File > Export Specified Packets, with "Displayed" selected, -- saves only
packets matching the current display filter
```

This is the standard way to hand off a relevant slice of a large capture — e.g. only the packets involving a suspicious host — without sharing an entire, possibly sensitive, full capture.

9.3 Printing and reporting

For written reports, screenshots of the Packet Details pane combined with Follow Stream output are usually clearer for a non-technical audience than raw packet lists — pair each with a plain-language description of what it shows and why it matters to the investigation's conclusion.

10

Reading Hex Dumps and the Packet Bytes Pane

10.1 Why read raw bytes at all

Wireshark's dissectors do almost all field decoding for you, but there are still real reasons to read the raw hex directly: verifying a dissector isn't misinterpreting a malformed or crafted packet, spotting hidden data a dissector doesn't parse, and building the low-level intuition that makes every other chapter in this book make more sense.

10.2 The Packet Bytes pane, field by field

The bottom pane shows three columns for the selected packet: a byte offset (far left, in hex), the raw bytes themselves (center, in hex), and their ASCII representation (right, non-printable bytes shown as dots). Clicking any field in the Packet Details pane highlights the exact bytes it came from in all three columns simultaneously.

A raw Ethernet/IP/TCP header, as Wireshark would show it

```
0000 ff ff ff ff ff ff 00 1a 2b 3c 4d 5e 08 00 45 00 0010 00 3c 1c 46 40 00 40 06
b1 e6 c0 a8 01 0a c0 a8 0020 01 01 c3 50 00 50 a1 2e 3f 4d 00 00 00 00 a0 02
```

10.3 Decoding a header by hand

Every protocol header is just a fixed sequence of byte offsets with defined meanings. Walking through the Ethernet header from the dump above:

Offset	Hex bytes	Meaning
0x00–05	`ff ff ff ff ff ff`	Destination MAC — all-Fs is the broadcast address
0x06–0B	`00 1a 2b 3c 4d 5e`	Source MAC address
0x0C–0D	`08 00`	EtherType — 0x0800 means the next layer is IPv4

Continuing into the IPv4 header that follows at offset 0x0E:

Byte(s)	Value	Meaning
1st nibble	`4`	IP version 4
2nd nibble	`5`	Header length: 5 x 4 = 20 bytes, no options
Protocol byte	`06`	Next layer is TCP (protocol number 6)
Bytes 12–15	`c0 a8 01 0a`	Source IP: 192.168.1.10
Bytes 16–19	`c0 a8 01 01`	Destination IP: 192.168.1.1

Why the protocol number matters

That single byte (`06` for TCP, `11` for UDP, `01` for ICMP) is how every IP stack in the world knows which layer to hand the payload to next — worth recognizing on sight when reading a raw dump.

10.4 Standalone hex tools outside Wireshark

Not every hex-dump task involves a full packet capture — these command-line tools cover raw file/byte inspection more generally.

Common hex-dump utilities

```
xxd capture.bin | head xxd -p capture.bin | tr -d '\n' -- plain hex, no
offsets/ASCII hexdump -C capture.bin | head od -A x -t x1z capture.bin | head
```

Reversing: hex back to binary

```
xxd -r -p hexfile.txt > output.bin
```

10.5 Exporting a packet as a hex dump from Wireshark

Right-click any packet and choose *Copy > … as Hex Dump* to get exactly the offset/hex/ASCII text shown in the Packet Bytes pane, ready to paste into notes, a ticket, or a report — useful for documenting a specific malicious packet without attaching a full capture.

10.6 Decode As: overriding protocol detection

Wireshark normally guesses the protocol from the port number, which breaks for services running on non-standard ports. Right-click a packet and choose *Decode As* to force Wireshark to parse it using a specific dissector regardless of port — for example, forcing port 8443 traffic to be interpreted as HTTP instead of being left as raw, undissected TCP.

When you'll need this

Malware C2 traffic is a common case — a sample might speak HTTP on a completely arbitrary port specifically to avoid casual inspection. Decode As recovers full field-level parsing the moment you know (or guess) what's actually being spoken underneath.

11

Command-Line Analysis with tshark

11.1 Why tshark

`tshark` is Wireshark's command-line counterpart, using the same capture engine, dissectors, and filter syntax — useful for scripting, remote analysis over SSH, processing captures too large to comfortably open in the GUI, or running unattended as part of a larger pipeline.

11.2 Basic usage

Capture and read

```
tshark -i eth0 tshark -r capture.pcapng tshark -i eth0 -w capture.pcapng tshark
-i eth0 -c 100
```

`-i` selects a capture interface, `-r` reads an existing file, `-w` writes a capture to disk, and `-c` stops after a fixed packet count.

11.3 Applying filters

Capture and display filters in tshark

```
tshark -i eth0 -f "tcp port 443" tshark -r capture.pcapng -Y "http.request"
tshark -i eth0 -f "host 10.0.0.5" -Y "dns"
```

The `-f` flag takes a BPF capture filter (same syntax as Wireshark's capture filters, applied at capture time); `-Y` takes a display filter (same syntax as the GUI's display filter bar, applied while reading). Both can be combined — capture broadly with `-f`, then narrow the view further with `-Y` on the same run.

11.4 Extracting specific fields

Field extraction for scripting/CSV output

```
tshark -r capture.pcapng -T fields -e ip.src -e ip.dst -e tcp.port tshark -r
capture.pcapng -Y "dns" -T fields -e dns.qry.name tshark -r capture.pcapng -T
fields -E header=y -E separator=, \-e frame.number -e ip.src -e ip.dst > out.csv
```

`-E header=y` adds a column header row, and `-E separator=,` switches the field separator to a comma — together turning tshark into a quick CSV export tool for any set of fields, no scripting required.

11.5 Output formats

Flag	Format
`-T text`	Default, human-readable summary (same as the GUI's one-line-per-packet view)
`-T fields`	Only the specific fields named with `-e`, ideal for scripting
`-T json`	Full packet detail as JSON, convenient for feeding into other tools
`-T ek`	Elasticsearch bulk-insert JSON, built for direct ingestion into an ELK stack
`-T pdml`	Full XML packet detail, the most complete machine-readable format
`-T psml`	Lightweight XML summary, faster to generate than PDML

JSON output, useful for jq pipelines

```
tshark -r capture.pcapng -Y "http.request" -T json \| jq
'.[]._source.layers."http.request.full_uri"'
```

11.6 Statistics from the command line

Quick stats without opening the GUI

```
tshark -r capture.pcapng -q -z io,phs tshark -r capture.pcapng -q -z conv,tcp
tshark -r capture.pcapng -q -z conv,ip tshark -r capture.pcapng -q -z http,tree
tshark -r capture.pcapng -q -z dns,tree
```

`-q` suppresses the normal packet-by-packet output so only the statistics report prints; `-z` selects which statistics module to run — `io,phs` is protocol hierarchy, `conv,*` is conversations by layer, and most GUI Statistics menu entries have a matching `-z` name.

11.7 Rolling captures and long-running collection

Ring buffer capture, for continuous unattended collection

```
tshark -i eth0 -b filesize:100000 -b files:10 -w rolling.pcapng
```

This keeps at most 10 files of roughly 100MB each, deleting the oldest as new ones are created — a practical way to keep a rolling window of recent traffic on a sensor without unbounded disk growth.

11.8 Combining tshark with standard Unix tools

Piping into grep, sort, and uniq

```
tshark -r capture.pcapng -T fields -e ip.src \| sort | uniq -c | sort -rn | head
```

Because `-T fields` output is just plain text, every standard command-line tool applies normally — this particular pipeline produces a quick ranked list of the most frequent source IPs in a capture, without writing a single line of dedicated code.

When to reach for tshark

Any time a capture is too large to comfortably load in the GUI, the analysis needs to run unattended or on a remote box over SSH, or the output needs to feed directly into another tool — tshark does everything Wireshark's GUI does, just without the panes.